

Module 001 – Self-Hosted Business Email

Neighborhood Blockchain Hub Build-Along

	Field	Value
Module Number		001
Module Title		Self-Hosted Business Email
System Built		yourname@yourbusiness.com email with auto-forwarders, auto-reply, and SMS inbox alerts
Host		Derrell Black, Blacktech Solutions Corp
Estimated Duration		75 minutes
Cost to Replicate		\$0 (uses existing web hosting)
Skill Level		Beginner
Prerequisites for Attendees		Domain name purchased; cPanel hosting active; Telegram on phone

Part A – What & Why (10 min)

The Problem

Gmail and Outlook scan every email you send and receive. They build profiles on your business relationships, pricing, and customer data. If Google suspends your account, you lose access to years of business communication instantly.

The Solution

Host your email on the same server that runs your website. You control the data. No third party reads your invoices, vendor quotes, or customer complaints.

What You Build

- support@yourbusiness.com – your main inbox
 - invoices@yourbusiness.com – auto-replies to anyone who emails it
 - vendors@yourbusiness.com – forwards straight to support@
 - A Python script on your Pi that checks support@ every few hours and texts you when money moves
-

Part B – The Build (50 min)

Step 1 – Create the Email Accounts (15 min)

Log into your web host's **cPanel**.

1. Find **Email Accounts** – click it
2. Create these four accounts:
 - support@yourbusiness.com
 - invoices@yourbusiness.com
 - vendors@yourbusiness.com
 - automation@yourbusiness.com (for tool logins like Zapier)
3. Set strong passwords. Store them in your vault – never in the browser.

Your email server settings (same for all four): - **Incoming (IMAP):** mail.yourbusiness.com | Port 993 | SSL - **Outgoing (SMTP):** mail.yourbusiness.com | Port 465 | SSL

Step 2 – Set Forwarders (10 min)

Still in cPanel:

1. Find **Forwarders** – click it
2. Add two forwarders:
 - vendors@yourbusiness.com – support@yourbusiness.com
 - invoices@yourbusiness.com – support@yourbusiness.com

Now every vendor email and every customer invoice lands in one place.

Step 3 – Enable Auto-Reply on Invoices@ (10 min)

1. In cPanel, go back to **Email Accounts**
2. Click **Manage** next to invoices@
3. Click **Auto-Reply**

4. Paste this message:

Thank you for reaching out.

Your message has been forwarded to our support team.
We will respond within 24 hours.

â€” [Your Business Name]

5. Save it

Anyone who emails your invoices address gets an instant professional response, even at 2 AM.

Step 4 â€” Test the Pipe (5 min)

Send a test email: - From your personal Gmail â†’ vendors@yourbusiness.com - Subject: â€œTest pipeâ€ - Check support@ inbox (via webmail in cPanel) â€” the test email should be there - Reply to the Gmail address from support@ â€” confirm it sends

Step 5 â€” Install the Inbox Robot on Your Pi (10 min)

On your Raspberry Pi, create a Python script that logs into support@, reads new emails, and texts you the important ones.

Create the file:

```
/home/[user]/.hermes/profiles/[your-name]/scripts/blacktech_email_robot.py
```

The script skeleton (fill in your credentials from your vault):

```
#!/usr/bin/env python3
import imaplib, email, json, os, re
from datetime import datetime, timedelta

# --- CONFIG (load from your vault, never hardcode real passwords) ---
IMAP_SERVER = "mail.yourbusiness.com"
IMAP_USER = "support@yourbusiness.com"
# Password stored in your local vault file â€” read it at runtime
IMAP_PASS = os.environ.get("EMAIL_PASS") or open("/path/to/your/vault/email_pass.txt").read().strip()
TELEGRAM_BOT_TOKEN = os.environ.get("TELEGRAM_BOT_TOKEN")
TELEGRAM_CHAT_ID = os.environ.get("TELEGRAM_CHAT_ID")

# Keywords that mean money moved
PAYMENT_KEYWORDS = ["payment received", "paid", "deposit", "invoice paid", "check cleared"]

# --- CONNECT ---
mail = imaplib.IMAP4_SSL(IMAP_SERVER)
mail.login(IMAP_USER, IMAP_PASS)
mail.select("inbox")

# --- SEARCH LAST 24 HOURS ---
yesterday = (datetime.now() - timedelta(days=1)).strftime("%d-%b-%Y")
status, messages = mail.search(None, f'(SINCE {yesterday})')

alerts = []
for num in messages[0].split():
    status, data = mail.fetch(num, "(RFC822)")
    msg = email.message_from_bytes(data[0][1])
    subject = msg["Subject"] or ""
    body = ""
    if msg.is_multipart():
        for part in msg.walk():
            if part.get_content_type() == "text/plain":
                body = part.get_payload(decode=True).decode(errors="ignore")
                break
    else:
        body = msg.get_payload(decode=True).decode(errors="ignore")

    full_text = f"{subject} {body}".lower()
    if any(k in full_text for k in PAYMENT_KEYWORDS):
        alerts.append(f"ðŸ’° {subject}")

# --- TEXT ME ---
if alerts:
    import requests
    msg = "\n".join(["New payments detected:" + alerts])
    url = f"https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendMessage"
    requests.post(url, json={"chat_id": TELEGRAM_CHAT_ID, "text": msg})
    print(f"[{datetime.now()}] Sent {len(alerts)} payment alerts")
else:
    print(f"[{datetime.now()}] No new payments")
```

Install the dependency:

```
pip install requests
```

Run it manually:

```
python3 blacktech_email_robot.py
```

Step 6 â€” Automate It (Cron) (5 min)

Run every 4 hours:

```
crontab -e
```

Add this line:

```
0 */4 * * * /usr/bin/python3 /home/[user]/.hermes/profiles/[your-name]/scripts/blacktech_email_robot.py >> /tmp/email_robot.log 2>&1
```

Save and exit. The Pi now checks your inbox 6 times a day and texts you when money hits.

Part C – Verify It Works (10 min)

Verification Script

Create `verify_email_system.py`:

```
#!/usr/bin/env python3
import imaplib, smtplib, sys

DOMAIN = "yourbusiness.com"
IMAP_SERVER = f"mail.{DOMAIN}"
SMTP_SERVER = f"mail.{DOMAIN}"
EMAIL = f"support@{DOMAIN}"
# Load password from your vault
PASS = open("/path/to/your/vault/email_pass.txt").read().strip()

def test_imap():
    try:
        mail = imaplib.IMAP4_SSL(IMAP_SERVER)
        mail.login(EMAIL, PASS)
        mail.select("inbox")
        print("â€¦ IMAP login works")
        return True
    except Exception as e:
        print(f"â€¦ IMAP failed: {e}")
        return False

def test_smtp():
    try:
        server = smtplib.SMTP_SSL(SMTP_SERVER, 465)
        server.login(EMAIL, PASS)
        print("â€¦ SMTP login works")
        return True
    except Exception as e:
        print(f"â€¦ SMTP failed: {e}")
        return False

def test_forwarder():
    print("âš¡ i, Forwarder test: manually send an email to vendors@ and check support@ inbox")
    return True # Requires human verification

if __name__ == "__main__":
    ok = all([test_imap(), test_smtp(), test_forwarder()])
    sys.exit(0 if ok else 1)
```

Run it:

```
python3 verify_email_system.py
```

Expected output:

```
â€¦ IMAP login works
â€¦ SMTP login works
âš¡ i, Forwarder test: manually send an email to vendors@ and check support@ inbox
```

Part D – Q&A + Homework (5 min)

Homework

1. Send yourself an email from Gmail to `vendors@` and confirm it lands in `support@`
2. Reply to that email from `support@` and confirm your Gmail receives it
3. Run `verify_email_system.py` and confirm all checks pass

Next Module Preview

Module 002 – Invoice Automation: Build a Python script that creates invoices in your accounting software and texts you when they go overdue.

Troubleshooting Quick Reference

Symptom

Most Likely Cause

Fix

â€œ535 authentication errorâ€	Wrong password	Check vault. Passwords are case-sensitive. No extra spaces.
â€œConnection refusedâ€	Wrong mail server	Must be mail.yourdomain.com, not your web hostâ€™s generic server
Emails not forwarding	Forwarder not saved	Re-check cPanel Forwarders list; delete and recreate
Auto-reply not sending	Auto-reply disabled	cPanel > Email Accounts > Manage > Auto-Reply â€” toggle ON
Pi script says â€œNo module requestsâ€	requests not installed	pip install requests
Texts never arrive	Wrong Telegram bot token or chat ID	Verify both values in your vault; test with a manual curl first

Decentralization Connection

Before	After	Next
Gmail reads your invoices	Only you read your invoices	Module 002: replace QBO with self-hosted invoicing (InvoiceNinja)
No alert when checks clear	Pi texts you within 4 hours	Module 003: auto-deposit crypto via BTCPay Server
Vendor emails scattered across apps	All vendor email centralized + auto-replied	Module 004: self-hosted project management (Plane or Taiga)

End of Module 001. Built and tested by Derrell Black, Blacktech Solutions Corp.